Hashcat Cheat sheet

Common usage	
Syntax	**Meaning**
hashcat -a <num> -m <#type> <hashfile> <dictfile>	Run hashcat in this <num> attack mode against this <hashfile> containing these <#htype> hashes with this <dictfile> list
-o <filename>	Output results to <filename>
attack modes	
-a 0	Dictionary attack
-a 1	Combinator attack - concatenates 2 wordlists
-a 3	Brute force attack
-a 6	Hybrid attack with dictionary and mask
-a 7	Hybrid attack with mask and dictionary
hash types	
-m 0	MD5
-m 10	MD5 with salt (hash:salt)
-m 20	MD5 with salt (salt:hash)
-m 100	SHA1
-m 110	SHA1 with salt (hash:salt)
-m 120	SHA1 with salt (salt:hash)
-m 1000	NTLM
-m 1100	Domain cached credentials
see https://hashcat.net/wiki/doku.php?id=example_hashes for full list	
--session=<name>	Name a hashcat session
--restore=<session>	Restore an interrupted session
--show	Show the cracked hashes from the potfile
--left	Show the uncracked hashes from the potfile

Hashcat Masks	
Syntax	**Meaning**
?l	lowercase alpha characters
?u	Uppercase alpha characters
?d	decimal characters
?h	lowercase hex characters
?H	Uppercase hex characters
?s	Symbols
?a	Equvalent to ?l?u?d?s

Why use masks?

The use of a mask is to reduce the password candidate keyspace to a more efficient one.

Let's say we want to crack the password: Julia1984

In traditional Brute-Force attack we require a charset that contains all upper-case letters, all lower-case letters and all digits (aka "mixalpha-numeric").

The Password length is 9 characters, so we have to iterate through 62^9 (13.537.086.546.263.552) combinations. Let's say we crack with a rate of 100M/s, this requires more than 4 years to complete.

In a mask attack we assume knowledge about humans and how they design passwords. The above password matches a simple but common pattern. A name and year appended to it.

We can configure the attack to try the upper-case letters only on the first position. It is very uncommon to see an upper-case letter only in the second or the third position.

With Mask attack we can reduce the keyspace to 52*26*26*26*26*10*10*10*10 (237.627.520.000) combinations. With the same cracking rate of 100M/s, this requires just 40 minutes to complete.

A mask for the above password pattern would look like this: ?u?l?l?l?l?d?d?d?d

or we can substitute the variable names with numbers:

1=l

2=u

3=d

4=s

In this case the mask would be:

?2?1?1?1?1?3?3?3?3

For more information, see https://hashcat.net/wiki/doku.php?id=mask_attack